

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. ШУХОВА»**
(БГТУ им. В.Г. Шухова)

Кафедра программного обеспечения вычислительной техники и автоматизированных систем



Лабораторная работа №4
по дисциплине: Теория информации
тема: Исследование кода Гилберта-Мура

Выполнил: ст. группы ПВ-221

Дорошенко Владимир Юрьевич

Проверили: Твердохлеб В. В.

Белгород 2024

ЗАДАНИЕ

- 1) Изучить принцип построения кода, используя пример в закреплённом файле «Пример. ЛБ4.pdf».
- 2) Построить обработчик, выполняющий компрессию по алгоритму Гилберта-Мура.
- 3) Создать генераторы данных, работающих как источники Хартли и Бернулли (в двоичном алфавите).
- 4) Сгенерировать 2 цепочки данных длиной 800 символов каждую (соответственно, порожденные по Бернуллевскому принципу и принципу Хартли).
- 5) Выбрать произвольную текстовую строку на русском языке длиной 100 символов, преобразовать в двоичный формат Unicode/ASCII (на выбор), пользуясь любым соответствующим онлайн-сервисом (на выходе имеем последовательность из 800 символов).
- 6) Построить коды Гилберта-Мура для каждой из последовательностей, для чего предварительно сегментировать каждую цепочку по 8 символов.
- 7) Вычислить полученные коэффициенты сжатия и величину дисперсии для каждой последовательности.
- 8) Результаты кодирования занести в сводную таблицу.

1) Построить обработчик, выполняющий компрессию по алгоритму Гилберта-Мура.

```
std::vector<bool> GetFloatNBits(  
    float number,  
    int n  
) {  
    std::vector<bool> res(n);  
    for (int i = 0; i < n; i++) {  
        number *= 2;  
        res[i] = int(number);  
        number -= int(number);  
    }  
  
    return res;  
}  
  
std::vector<FanoCode> GetMooreHilbertCode(const std::unordered_map<wchar_t, int>&  
counters) {  
    if (counters.empty()) {  
        return {};  
    }  
  
    if (counters.size() == 1) {  
        return { {counters.begin()->first, 1, std::vector{true}} };  
    }  
  
    std::vector<FanoCode> codes;  
    std::vector<float> d;  
    std::vector<float> sigma;  
    size_t total = 0;  
  
    for (auto& i : counters) {  
        codes.push_back({ i.first, i.second });  
        total += i.second;  
    }  
  
    for (int i = 0; i < codes.size(); i++) {  
        double p = static_cast<float>(codes[i].count) / total;  
  
        if (i == 0) {  
            d.push_back(0);  
        }  
        else {  
            double prev_p = static_cast<float>(codes[i - 1].count) / total;  
            d.push_back(d[i - 1] + prev_p);  
        }  
  
        sigma.push_back(d[i] + p / 2);  
        int length = ceil(-std::log2(p)) + 1;  
        codes[i].code = GetFloatNBits(sigma[i], length);  
    }  
  
    return codes;  
}
```

2) Построить обработчик, выполняющий компрессию по алгоритму Гилберта-Мура.

```
std::wstring HartliGenerator(  
    int n,  
    int n_bits  
) {  
    std::string t;  
    std::random_device rd;  
    std::mt19937 gen(rd());  
    std::uniform_int_distribution<unsigned char> d(0, std::min((1 << n_bits) - 1,  
127));  
    for (int i = 0; i < n; i++) {  
        t.push_back(d(gen));  
    }  
  
    return utf8_to_utf16(t);  
}
```

3) Создать генераторы данных, работающих как источники Хартли и Бернулли (в двоичном алфавите).

```
std::string BernoulliGenerator(  
    int n,  
    float p  
) {  
    std::string t;  
    std::random_device rd;  
    std::mt19937 gen(rd());  
    std::bernoulli_distribution d(p);  
    for (int i = 0; i < n; i++) {  
        t.push_back(d(gen) + '0');  
    }  
  
    return t;  
}
```

4) Построить коды Гилберта-Мура для каждой из последовательностей, для чего предварительно сегментировать каждую цепочку по 8 символов.

Алгоритм Гилберта-Мура для источника Хартли

Код:

<!!> = 11111110
<+> = 11111100
<↑> = 1111100
<4> = 11110100
<♠> = 11110001
<M> = 11101111
<↑> = 11101100
<♣> = 11101010
<{> = 1110010
<W> = 11011111
<6> = 11011101
<Q> = 11011010
<L> = 11011000
<,> = 1101010
<1> = 1100111
<V> = 01010000
<5> = 01001110
<p> = 01001011
<0> = 01001000
<\$> = 0100010
<_> = 00001110
<x> = 01000001
<=> = 0011110
<n> = 00110111
<3> = 00110100
</> = 00110001
> = 00101010
<H> = 00100111
<S> = 00100101
<:> = 00100010
<u> = 01011101
<f> = 00000001

<<> = 00000011
<w> = 100111
<☺> = 10100101
<@> = 00000110
<♥> = 00101111
<y> = 00101100
<z> = 00011101
<F> = 00001000
<~> = 00001011
> = 01111110
<C> = 1000001
<h> = 00010000
<-> = 00011010
<K> = 00010011
<л> = 00010101
<\> = 00011000
<7> = 00011111
<r> = 011010
<s> = 01010011
<8> = 01010101
<[> = 0101100
<X> = 0110001
<T> = 0111001
<)> = 01110111
<.> = 0111101
<|> = 10000110
<♠> = 10001000
<т> = 10001011
<O> = 10001110
<R> = 10010000
<`> = 10010011
<z> = 10010101
<#> = 10011000
<(> = 1010101
<c> = 10101111
<'> = 1011001

<

> = 10110111

<E> = 10111001

<e> = 1011111

<> = 11000011

 = 1100011

<}> = 11001011

Исходное сообщение в символьном представлении:

uX♠rC.T|R`¶#) rxspV80[\$z=3/r[\$♥yHwn(X'

l,({↑}+!!rwS.:7Z-\e=ЉBKTh_~F@<{↑f

Закодированное сообщение в двоичном представлении:

01011101011000110001000011010100000101111010111001100001101001000010010011100
01011100110000111011101101001000001010100110100101101010000011000101111110010
10101010010000101100010001010010101001111000110100001100010110100101100010001
00010111100101100001001111001110011011110101010110001101100110110111100011101
01111111000011110011110111001110101010111111010010111000111001111101100010011
1110110100100111011011101110111111010111111001011101100001111010000011110101
011110001101100111101111101010111110100001010101100111110101010101111001011
1110011001011111111001111110011010100111001001010111101001000100001111100011
10100011010000110001011111001111000010101110001100010011011100100010000000011
1000001011000010000000001100000000111110010111110000000001

Алгоритм Гилберта-Мура для источника Бернулли

Код:

<←> = 1111110
<~> = 1111011
<C> = 11110001
<6> = 1110111
<→> = 11101010
<n> = 111000
<3> = 11011101
<л> = 11011010
<I> = 11011000
<N> = 1101001
<!!> = 11001110
<9> = 11001011
<s> = 1100011
<O> = 1011111
<}> = 1011100
<G> = 10110100
<o> = 101010
<7> = 1010000
<v> = 1001110
<;> = 1001010
<?> = 1000111
<z> = 10001011
<\> = 10001000
<u> = 1000001
<:> = 01111110
<x> = 01111100
<=> = 0111100
<f> = 00100101
<5> = 00011010
<▼> = 00100111
<m> = 00010000
<†> = 00100010
<1> = 00010011
<1> = 0001011
<k> = 0001111
<>> = 0000110

<[> = 0111001
<a> = 00001000
<-> = 00000110
<g> = 00000011
<*> = 00000001
<U> = 00101010
<<> = 00101111
<w> = 00101100
<◄> = 00110001
<K> = 00111001
<.> = 00110111
<i> = 00110100
<△> = 0011110
<j> = 010100
<@> = 01000001
<{> = 010001
<_> = 0101100
<|> = 0110000
<L> = 0110011
<W> = 0110110

Исходное сообщение в символьном представлении:

;7vu|o[W_\u]WoW{=G}OxsN?Nлn—|{96znn~6s:=]←uΔK3@?.In;o◄{i<oΔLU▼]←f{o}~!!O‡}kNs5
vlow[1|m]C>l>a-Lkg7*~;_

Закодированное сообщение в двоичном представлении:

10010101010000100111010000010110000101010011100101101100101100100010001000001
010100011011010101001101100100010111100101101001011100101111011110011000111
1010011000111110100111011010111000111010100110000010001110010111101111000101
11110001110001111011110111110001101111100111100010100111110100000100111100
0111001110111010100000110001110011011110110001110001001010101010001100010100
01001101000010111110101000111100110011001010100010011101010011111100010010101
00011010101011100111101111001110101111100100010101110000011111101001110001100
01101010011100001011101010001011000111001000101101100000001000001010011110001
00001100001001100001100000100000000110011001100011110000001110100000000000111
1101110010100101100

Алгоритм Гилберта-Мура для сообщения

Код:

<о> = 11110
<ы> = 1110011
<ь> = 1110000
<л> = 11011101
<е> = 110101
<т> = 11000
<г> = 1011010
<р> = 10100
<, > = 10011010
< > = 10001
<д> = 01111001
<н> = 011100
<а> = 01100
<с> = 010100
<м> = 00000001
<щ> = 00000011
<й> = 00000110
<ц> = 00001000
<и> = 00010
<ю> = 0010001
<к> = 0010101
<у> = 0011000
<п> = 0011011
<в> = 010000
<я> = 00111100
<К> = 01001011
<з> = 01001000

Исходное сообщение в символьном представлении:

Картельные стоворы не допускают ситуации, при которой активно развивающиеся страны третьего мира при

Закодированное сообщение в двоичном представлении:

01001011011001010011000110101110111011110000011100111001111010110001010100101
10101111001000011110101001110011100010111001101011000101111001111100011011001
10000101000010101011000010001110001000101010000010110000011000011000000100000
01000010100110101000100110111010000010100010010101111101100011110101001111000
00011010001011000010101110000001001000001110011110100011010001100010010000100
000001001000001100001000100000011000101110101010100001111001000101010011000101
00011000111001110011100011100010100110101110001110000110101101101011110100010
00000010001010100011001000100110111010000010

5) Вычислить полученные коэффициенты сжатия и величину дисперсии для каждой последовательности.

Алгоритм Гилберта-Мура для источника Бернулли

Длина исходного сообщения в битах: 800

Длина закодированного сообщения в битах: 712

Коэффициент сжатия: 1.1236

Средняя длина кодового слова: 7.12

Дисперсия: 0.4656

Алгоритм Гилберта-Мура для сообщения

Длина исходного сообщения в битах: 800

Длина закодированного сообщения в битах: 583

Коэффициент сжатия: 1.37221

Средняя длина кодового слова: 5.83

Дисперсия: 1.0811

Алгоритм Гилберта-Мура для источника Хартли

Длина исходного сообщения в битах: 800

Длина закодированного сообщения в битах: 749

Коэффициент сжатия: 1.06809

Средняя длина кодового слова: 7.49

Дисперсия: 0.4099